

Exercices : Structures de données (T ^{le} NSI)	Séance 2
Objectifs : Connaître et utiliser les concepts de la programmation orientée objet (objet, classe, attribut, méthode, constructeur, instance)	

Exercice 1

Définir une classe **Date** pour représenter une date, avec trois attributs **jour**, **mois** et **annee**.

1.1 Écrire son constructeur.

1.2 Ajouter une méthode `__str__` qui renvoie une chaîne de caractères de la forme "8 mai 145".

On pourra se servir d'un attribut de classe qui est un tableau donnant les noms des douze mois de l'année. Tester en construisant des objets de la classe Date puis en les affichant avec print.

1.3 Ajouter une méthode `__lt__` qui permet de déterminer si une date **d1** est antérieure à une date **d2** en écrivant **d1 < d2**. La tester.

Exercice 2

Représente le diagramme de classes du projet de gestion du parking avec la classe **Voiture** et la classe **Parking**.

Exercices : Structures de données (T ^{le} NSI)	Séance 2
Objectifs : Connaître et utiliser les concepts de la programmation orientée objet (objet, classe, attribut, méthode, constructeur, instance)	

Exercice 1

Définir une classe **Date** pour représenter une date, avec trois attributs **jour**, **mois** et **annee**.

1.4 Écrire son constructeur.

1.5 Ajouter une méthode `__str__` qui renvoie une chaîne de caractères de la forme "8 mai 145".

On pourra se servir d'un attribut de classe qui est un tableau donnant les noms des douze mois de l'année. Tester en construisant des objets de la classe Date puis en les affichant avec print.

1.6 Ajouter une méthode `__lt__` qui permet de déterminer si une date **d1** est antérieure à une date **d2** en écrivant **d1 < d2**. La tester.

Exercice 2

Représente le diagramme de classes du projet de gestion du parking avec la classe **Voiture** et la classe **Parking**.

Correction

Exercice 1

```
class Date :
    """Une classe pour représenter une date"""
    def __init__(self, jour, mois, annee):
        self.jour = jour
        self.mois = mois
        self.annee = annee

    mois_annee = ["janvier", "février", "mars", \
                  "avril", "mai", "juin", "juillet", \
                  "août", "septembre", "octobre", \
                  "novembre", "décembre"]

    def __str__(self) :
        date_format = str(self.jour) + " "
        date_format += Date.mois_annee[self.mois-1]
        date_format += " " + str(self.annee)
        return date_format

    def __lt__(self, d2):
        """Retourne True si la date est antérieure à d2

        ie. si l'année est antérieure
           ou dans le cas où c'est la même année : le mois est antérieur
           ou sinon si le jour est antérieur
        """
        # and a une priorité plus forte que or
        return self.annee < d2.annee or self.annee == d2.annee and \
            (self.mois < d2.mois or self.mois == d2.mois and self.jour < d2.jour)
```

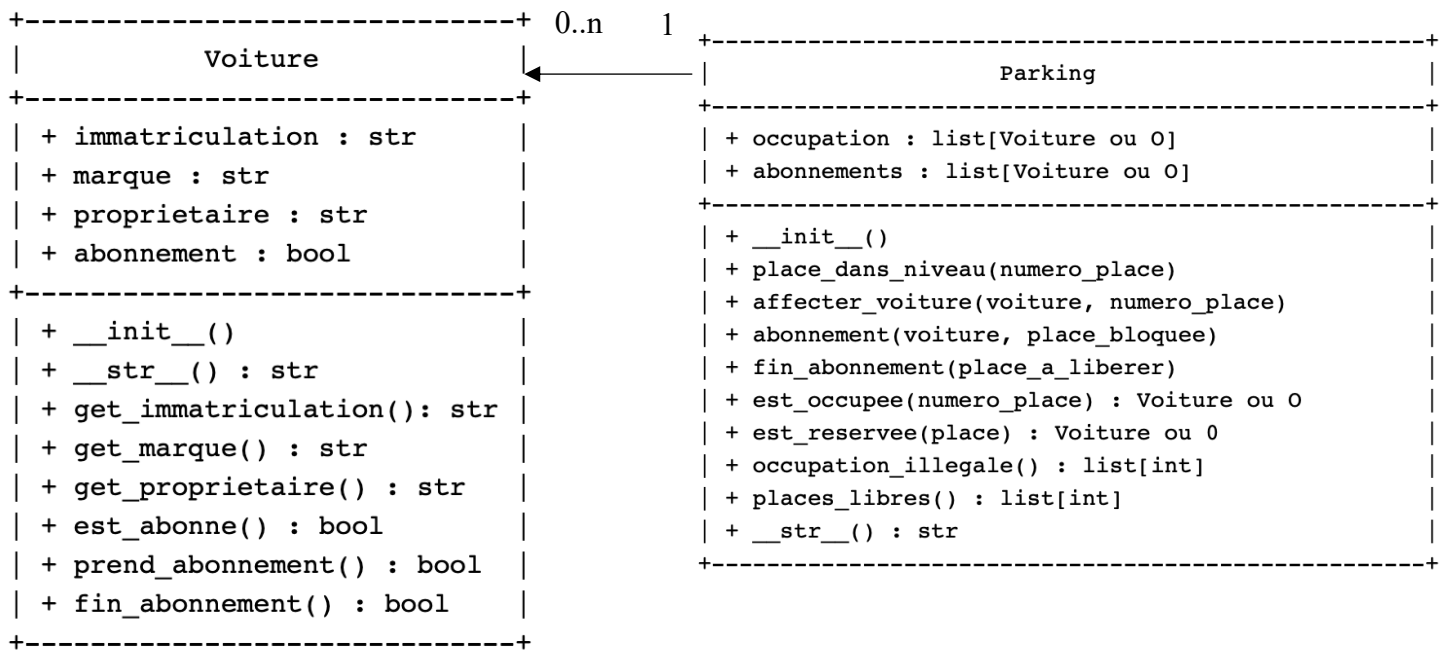
```
date1 = Date(1,2,2014)
date2 = Date(18,2,2014)
print(date1)
print(date2)
if date1 < date2:
    print(f"d1 soit {date1} est antérieure à d2 soit {date2} ")
else :
    print(f"d2 soit {date2} est antérieure à d1 soit {date1}")
```

1 février 2014

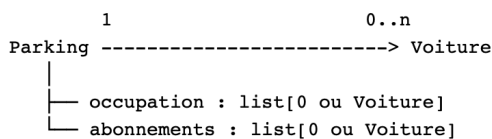
18 février 2014

d1 soit 1 février 2014 est antérieure à d2 soit 18 février 2014

Exercice 2



Relation entre les classes



Interprétation :

- Un objet **Parking** possède une liste `occupation` qui contient 0 si la place est libre ou une **Voiture**.
- Un objet **Parking** possède également une liste `abonnements` qui contient 0 si la place n'est pas réservée ou une **Voiture** ayant réservé cette place.
- Un parking peut contenir plusieurs voitures mais une voiture est garée dans un seul parking à la fois.

Interprétation :

- Un objet **Parking** possède une liste `occupation` qui contient 0 si la place est libre ou une **Voiture**.
- Un objet **Parking** possède également une liste `abonnements` qui contient 0 si la place n'est pas réservée ou une **Voiture** ayant réservé cette place.
- Un parking peut contenir plusieurs voitures mais une voiture est garée dans un seul parking à la fois.